Assignment0 环境配置与测试脚本的使用

本次作业的目的是为了让同学们完成编程环境的配置，熟悉完成作业、测试自己的答案并最终提交作业的过程，在之后完成作业时减少不必要的障碍。在跟随本文档完成环境配置的过程中遇到任何困难都可以咨询自己的责任助教。

CodeBlocks的安装与使用

请参见“CodeBlocks安装和使用.docx”文件，以在codeblocks中成功运行了一次程序为准。

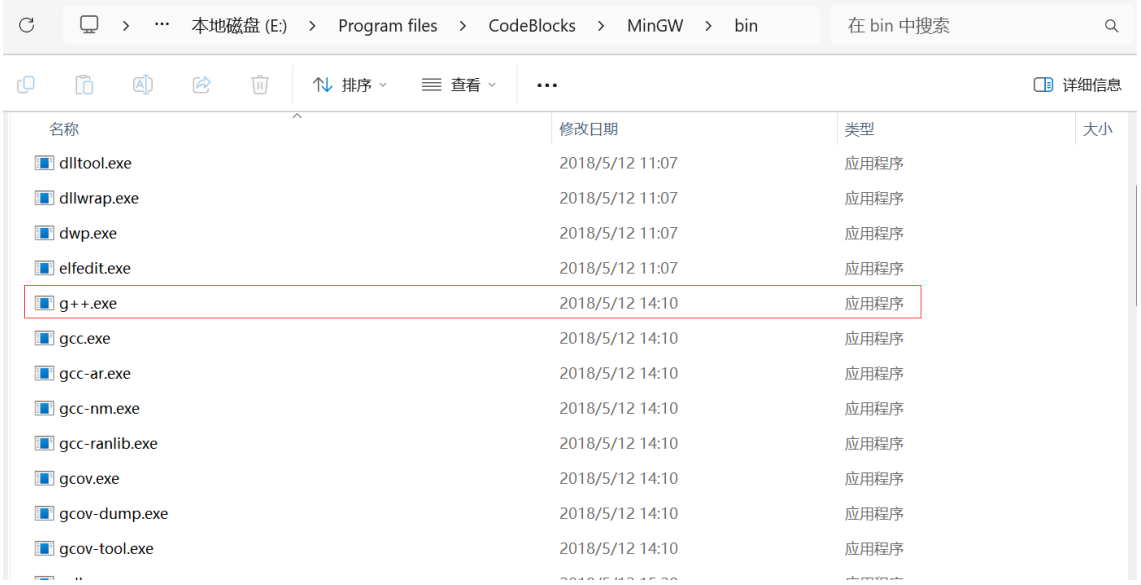
测试脚本所需的环境配置

我们使用的测试脚本`judger.py`使用Python实现，C++程序的编译需要g++编译器。因此使用这一脚本需要将g++和Python添加到系统的环境变量中。

配置g++到环境变量

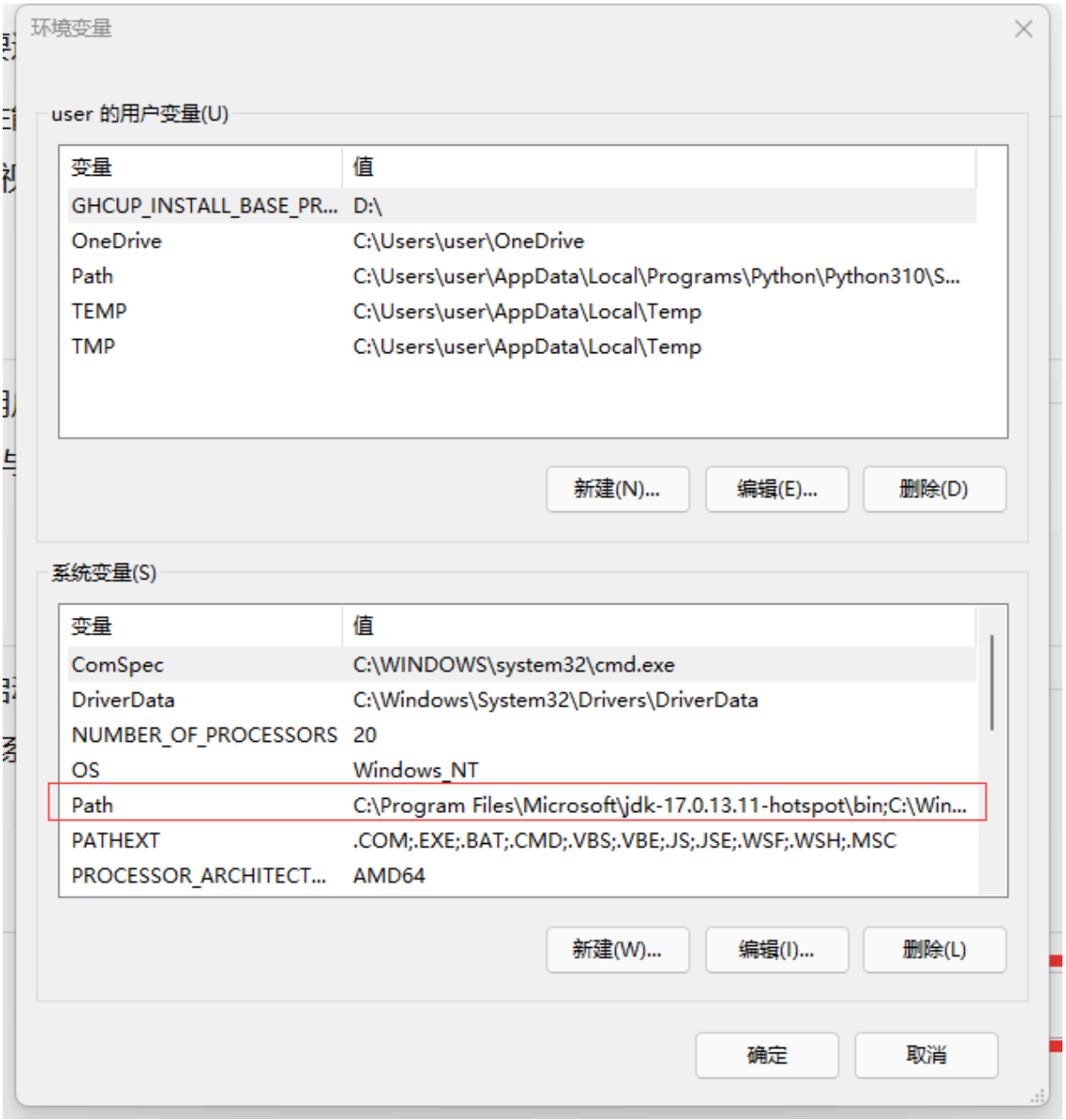
首先打开`windows powershell`或者`cmd`,输入`g++ -v`命令，如果提示“不是内部或外部命令也不是可运行的程序”，则说明`g++`未被配置到环境变量中。

1. 首先找到`g++.exe`程序，位置就在CodeBlocks的安装目录下（如果你没有使用CodeBlocks，请自行寻找路径)。例如如果将CodeBlocks安装在如下路径`E:\Program files\CodeBlocks`,那么`g++.exe`所在的路径就是`E:\Program files\CodeBlocks\MinGW\bin`,可以先打开对应的路径确认一下,例如：

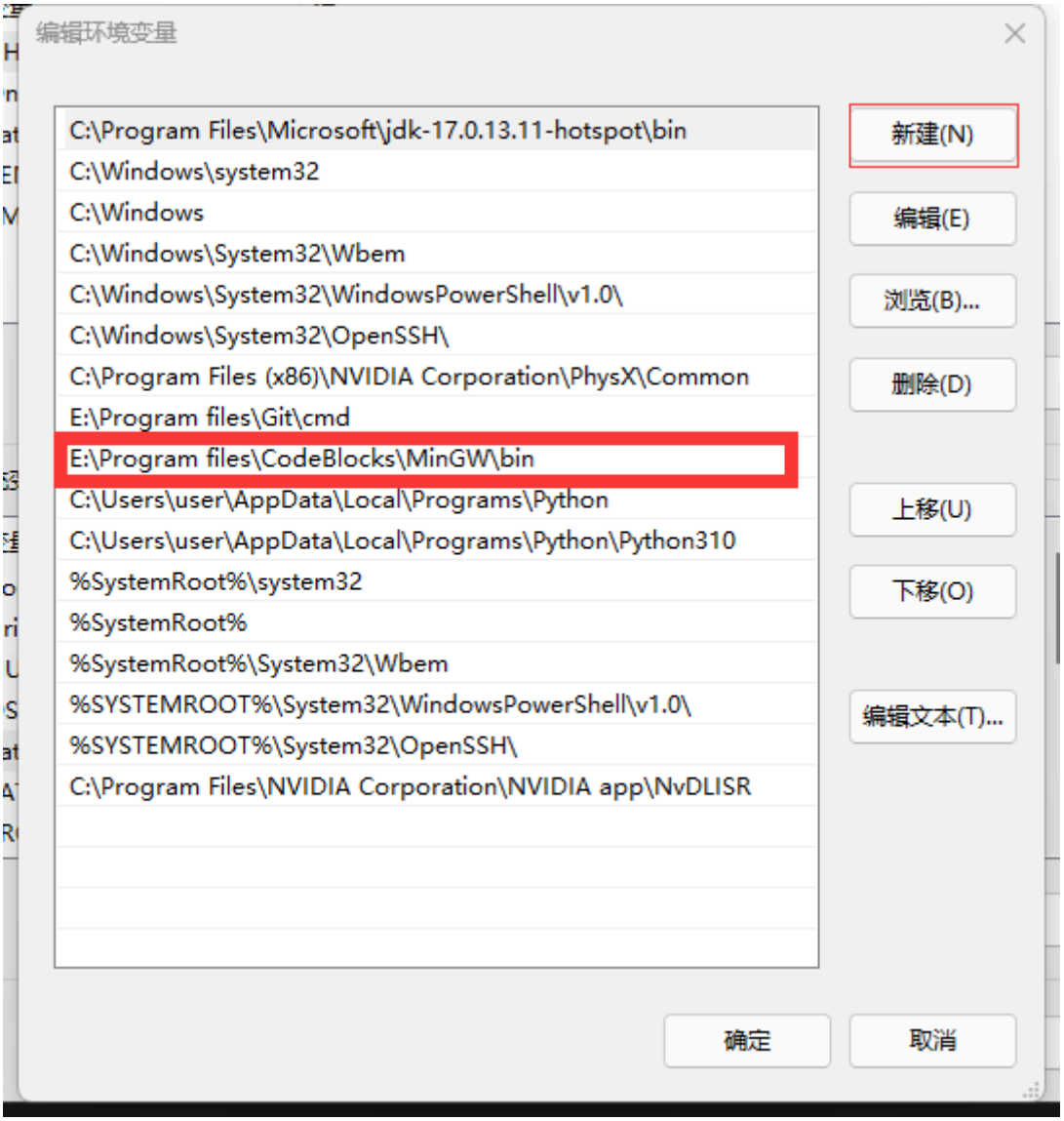


2. 打开环境变量设置。直接在Windows的搜索栏中搜索“编辑系统环境变量”，进入如下页面：

点击“环境变量”，再双击“系统变量”中的“Path”。



进入如下页面，点击“新建”并将上面找到的路径复制进去，得到如下效果：



最后一路点击“确定”完成设置。

3. 测试。配置完成后**请关闭之前打开的**`powershell`**或者**`cmd`**，再次打开一个新的命令行**并输入`g++ -v`，如果成功，会在输出的最后显示`g++`的版本信息如下：

```
Thread model: posix
gcc version 8.1.0 (x86_64-posix-seh-rev0, Built by MinGW-W64 project)
PS D:\Study\2025CPP>
```

配置Python到环境变量

由于我们的测试脚本使用Python，也需要Python在环境变量中。类似地打开`windows powershell`或者`cmd`,输入`python`命令，如果成功运行，说明Python已经在环境变量中。如果没有请按上述相同的方法将其添加到环境变量的“Path”中。

作业文件结构和测试脚本的使用方法说明

文件结构

我们给出的作业文件夹具有以下的文件结构：

```
|- 1_hello
|   |- main.cpp
|
```

```
|- 2_A+B
|   |- main.cpp
|
|- data
|   |- 1_hello
|       |- ....
|   |- 2_A+B
|       |- ....
|
|- judger.py
|
|- judger_batch.py
```

其中以数字开头的是各个题目的工程所在文件夹，每个文件夹中包含一个main.cpp作为源文件。本次我们提供的是**完整**的代码实现，在之后的作业中不会提供实现，你的主要任务就是完成每个题目的源代码。

`data`文件夹包含的是我们提供的**一部分**测试案例，在每一个子文件夹中以`n.in`,`n.out`的形式保存。你可以通过记事本或者其他文本编辑器打开并查看这些案例，不过不要改动其中的内容，因为我们的自动测试脚本需要使用这些案例。

题面

本课程中绝大多数的编程题都采用输入输出测试，即在给定的输入下，判断你写的程序的输出结果是否和标准的输出一致。本次作业包括两道非常简单的示例题。

1_hello

本题要求在输入的文本前加上“Hello,”，后面加上“!”之后输入，例如：
输入

```
world
```

输出

```
Hello,world!
```

2_A+B

本题要求将输入的两个整数相加并输出结果，例如：
输入

```
993 7
```

输出

```
1000
```

测试脚本的使用

剩下的两个Python脚本用于测试你的程序。其中`judger.py`使用时需要详细指定源代码和输入输出文件的位置,且只能一次测试一个样例。而`judger_patch.py`用于执行批量测试，支持省略部分参数。

首先讲解最简单也是最常用的测试方法，假设你已经在`1_hello`文件夹中完成了第1题的`main.cpp`文件，想要通过测试案例检查自己的实现是否正确，在`judger_patch.py`所在的本目录下打开命令行并输入`python judger_batch.py -T 1_hello`，可以得到如下结果。

```
PS D:\Study\2025CPP\cs1604\assignments\assignment0> python .\judger_batch.py -T 1_hello
[INFO] Config: Namespace(task='1_hello', input_dir='data\\1_hello', standard_dir='data\\1_hello', source_dir='1_hello',
timeout=2)
[TEST POINT 1]:
[INFO] Correct
[SCORE] 10
[TEST POINT 2]:
[INFO] Correct
[SCORE] 10
[TEST POINT 3]:
[INFO] Correct
[SCORE] 10
```

如果显示`g++`或者`python`找不到，说明环境配置没有成功。上图中的信息显示第一个题目`1_hello`的测试结果，三个测试案例均通过。输入中的`-T`后面跟着的是被测试题目的名字，如果想要测试第二题，那么输入`python judger_batch.py -T 2_A+B`即可。

注意以上最简单的使用格式只在你**使用默认的文件结构**时才可以工作，`judger_batch.py`的完整输入格式为：

```
python judger_batch.py -T ${task name} -I ${input folder} -O ${output folder} -S
${source folder}
```

其中`-I`后面跟着的是输入文件路径，`-O`后面跟着的是输出文件路径，`-S`后面的是要评测的源代码所在的路径。如果只填写`-T`,那么脚本会自动在上述默认提供的文件结构下寻找对应的源文件和测试文件。如果你没有按照我们提供的结构放置这些文件，那么你可以通过完整命令自行指定。脚本会自动对我们提供的所有样例进行批量测试。

如果你想要测试单独的一个样例，或者是你自己编写的新样例，那么可以使用`judger.py`,其完整格式为：

```
python judger.py -T ${task name} -I ${input file} -O ${output file} -S ${source
folder}
```

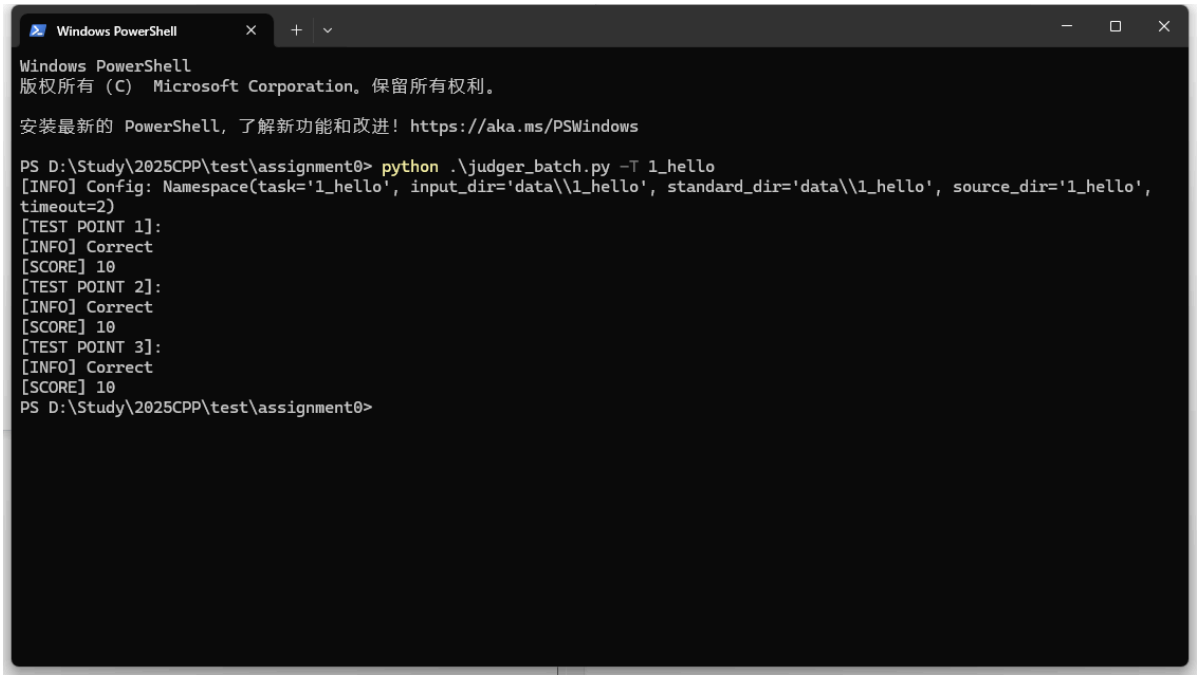
注意这里的区别是用`-I`和`-O`指定的不是测试文件所在的路径而是单个文件本身。用`judger.py`测试第一题的第一个样例的输入和效果如下：

```
PS D:\Study\2025CPP\cs1604\assignments\assignment0>
 python .\judger.py -T 1_hello -I data/1_hello/1.in
 -O data/1_hello/1.out -S 1_hello
[INFO] Config: Namespace(task='1_hello', input_file
='data/1_hello/1.in', standard_file='data/1_hello/1
.out', source_dir='1_hello', timeout=2)
[INFO] Correct
[SCORE] 10
```

作业的上传

一般来说，作业提交的格式为：

```
<your student number>.zip
|- 1_hello_sjtu
|   |- main.cpp
|
|- 2_A+B
|   |- main.cpp
```

我们推荐的完成作业的流程是在CodeBlocks中写完所有程序，将写完的`main.cpp`文件放到对应的文件夹下并通过了`judger_batch.py`的测试。此时只需要多选选中这些文件夹将它们压缩起来，之后**以你的学号命名**，就得到了满足要求的上传形式。每个题目的文件夹中可能包括`main.cpp`之外的其他文件，我们*建议*你将这些文件删除再压缩，我们在评测时只会评测以`.cpp`结尾的源文件。但请注意**不要**先把分别的题目文件夹放在一个以你的学号名字命名的文件夹之后再打包，这样有可能导致上传格式错误，增加助教批改时的工作量。

本次作业由于代码都是给定的，只需要在Canvas**上传你成功使用judger测试脚本的截图**即可，例如下图：